

Image Segmentation

CS 4243 Computer Vision & Pattern Recognition

Angela Yao

Changelog:

- 10. Mean-shift clustering (not part of syllabus this year)
- 40. Diagram not consistent with formula.

Recap & Outline

Last Week

- Image Classification
- Neural Networks & MLPs
- Convolutional Neural Networks
- Object Detection
- CNN-Based Object Detection

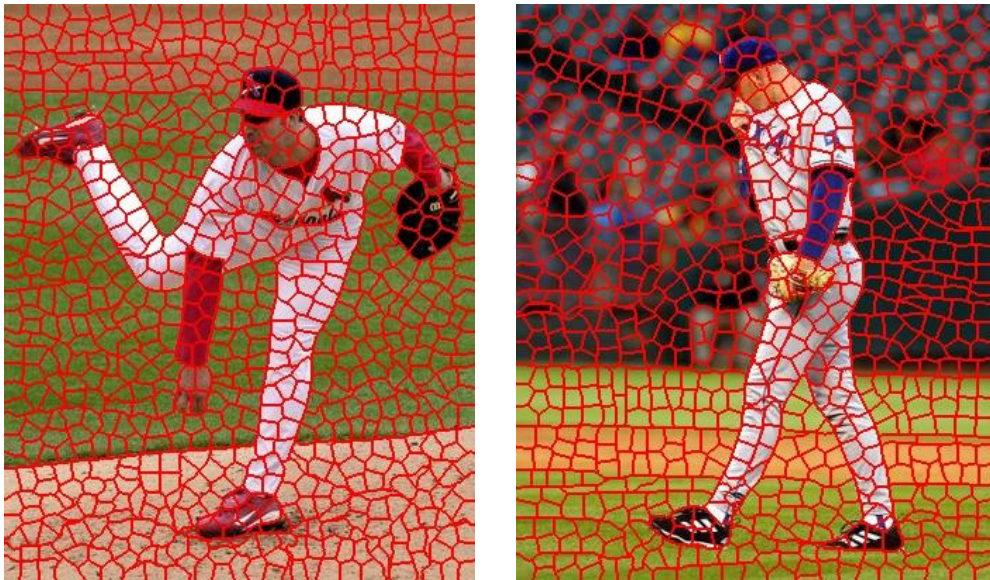
Today's Lecture

- Goal of segmentation
- Segmentation as a clustering problem
- K-means clustering for segmentation
- SLIC super-pixelling
- Segmentation with neural networks
- Promptable segmentation models

Goals of Segmentation

- Separate image into coherent regions or “objects”
- Group together similar pixels for efficiency of further processing

“superpixels”

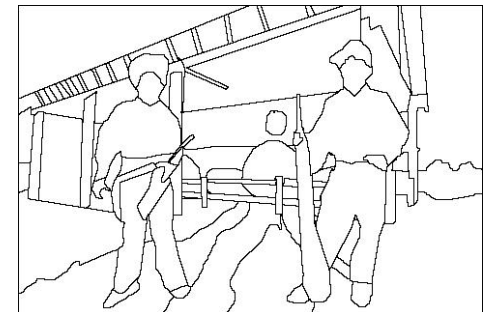
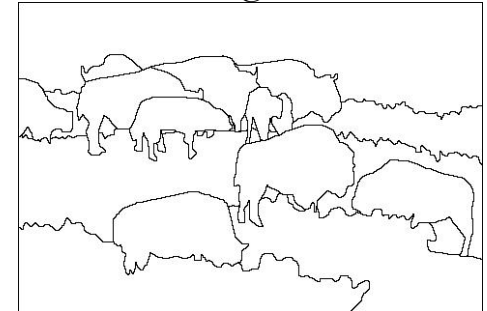


Ren & Malik. [Learning a classification model for segmentation](#), ICCV 2003.

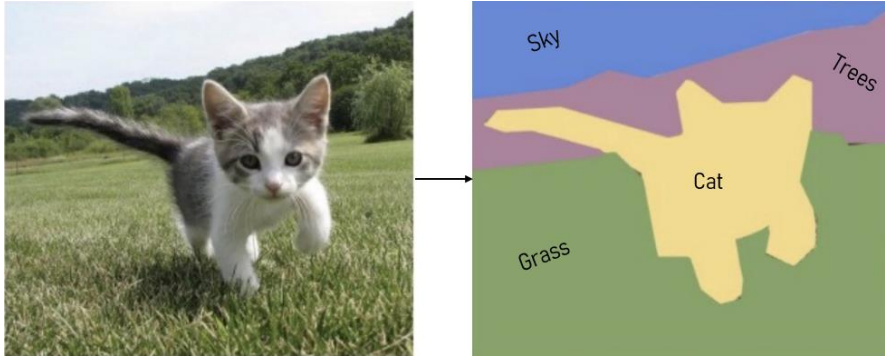
Image



Human segmentation

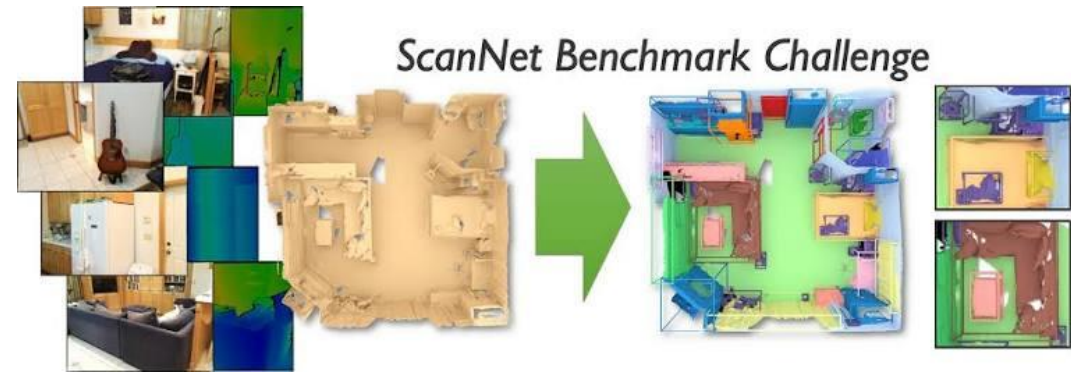


Segmenting Images, Video, Point Clouds, etc...



<https://datahacker.rs/020-overview-of-semantic-segmentation-methods/>

Image Segmentation



[ScanNet | Richly-annotated 3D Reconstructions of Indoor Scenes](#)

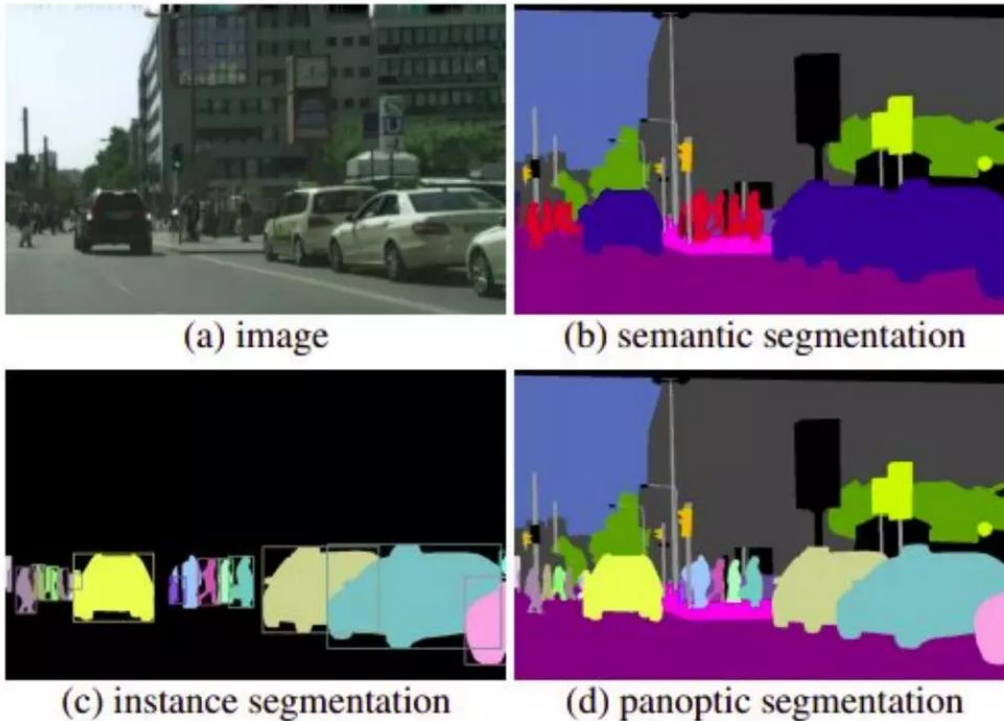
3D (Point Cloud) Segmentation



[SAM 2: Segment Anything Model 2 | Ultralytics](#)

Video Segmentation

Types of Image Segmentation



An image may have many different segmentations depending on the task, the algorithm and parameters settings.

The “correct” segmentation depends on the objective and application at hand.

Which Pixels Belong Together?

Every segmentation method must define a notion of **pixel affinity**:
what evidence makes two pixels belong to the same region?

Similar colour



Different objects

Same object



**Different colours
and materials**

Same category



All cars

- Similar color
- Similar texture
- Spatially connected
- Same object
- Same semantic category
- Selected by a user prompt

From Pixels to Regions

Affinity-based segmentation



1. **Describe pixels:** colour, texture, position
2. **Compare:** choose a similarity or distance
3. **Group:** cluster pixels into regions

k-means, SLIC

Learning-based segmentation



1. **Provide examples:** images and target masks
2. **Learn features:** train a segmentation network
3. **Predict:** segment a new image

FCN, U-Net, Mask R-CNN, SAM

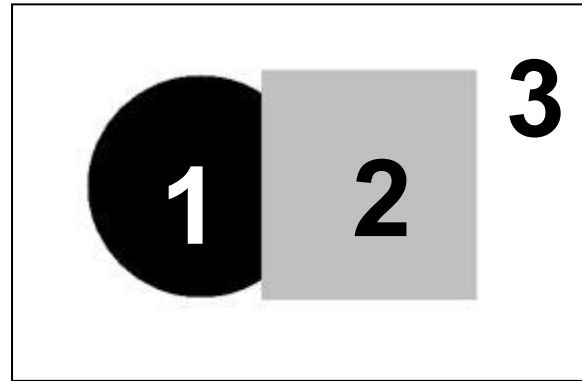
Grouping in Feature Space

Grouping for clustering
K-Means Clustering

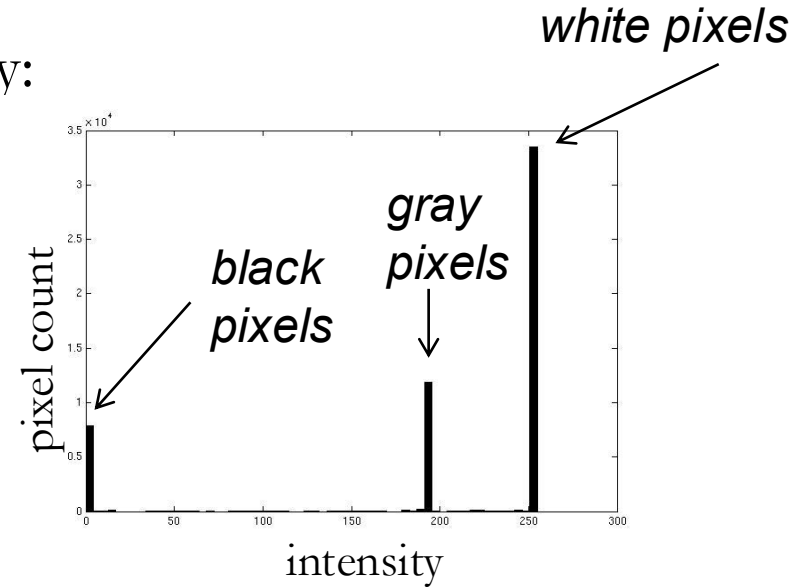
A Toy Example on Segmentation

Label every pixel based on its primary intensity:

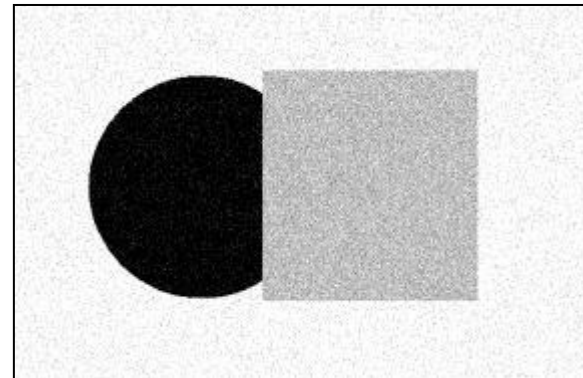
*Segmentation
based on intensity.*



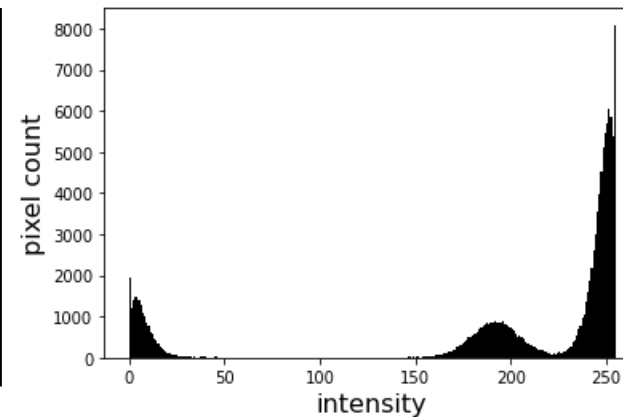
input image



*What if the image
was not so simple?*



input image



“Segmentation” = Clustering ?

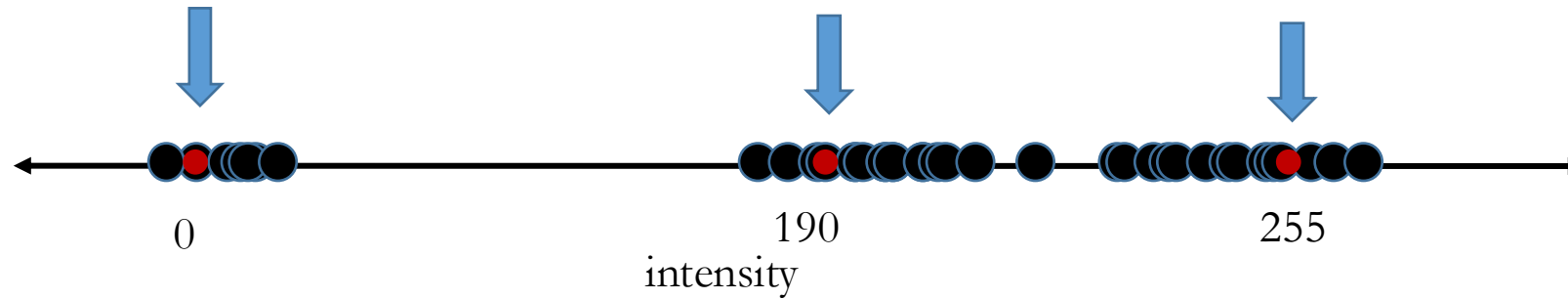
Clustering: group similar data points together and represent them with a single token

Two Key Challenges:

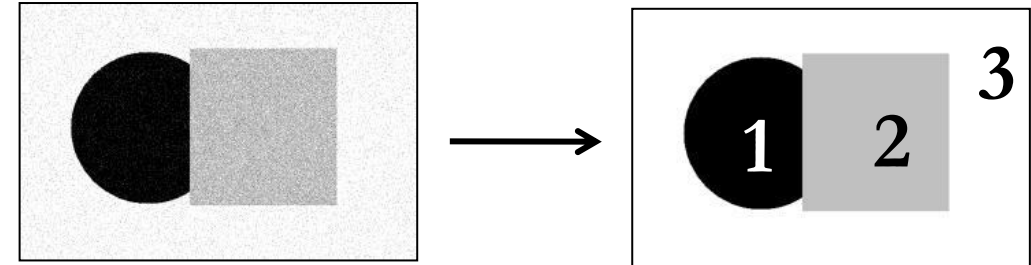
1. What makes two points similar or different?
 - Representing data (pixels) with features
 - Similarity/distance measure between the features
2. How do we compute an overall grouping from pairwise similarities? (Which clustering algorithm?)
 - K-means: iteratively re-assign points to the nearest cluster center
 - ~~• Mean-shift clustering: estimate modes of a probability density function~~

For purposes of our lecture today, we will consider only simple features such as pixel intensity or colour, and use a simple sum of squared differences (SSD).

Clustering



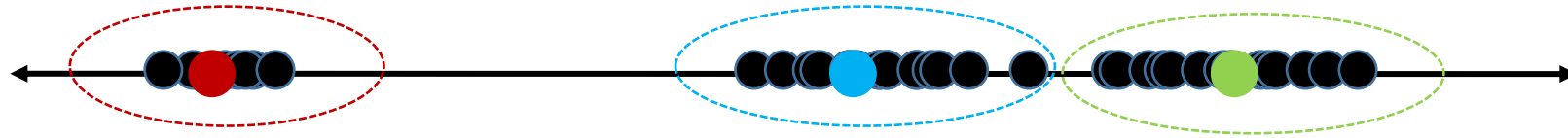
- Goal: choose three “centers” as **representative** intensities, and label every pixel based on the nearest centers
- The best cluster centers minimize squared difference between all points and their nearest cluster center.



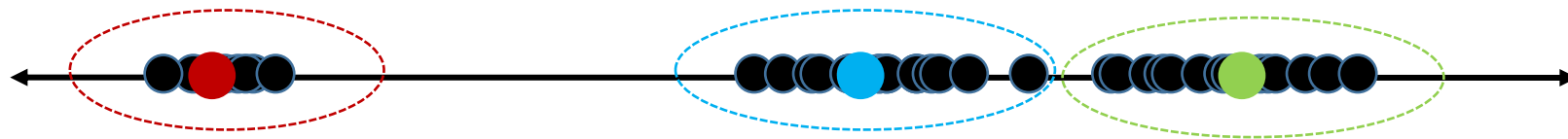
Clustering

With this objective, it is a “chicken and egg” problem:

- If we knew the **cluster centers**, we could allocate points to groups by assigning each to its closest center.



- If we knew the **group memberships**, we could get the centers by computing the mean per group.



k-Means Clustering

Basic idea: randomly initialize the k cluster centers, and iterate between assigning membership and computing cluster centers.

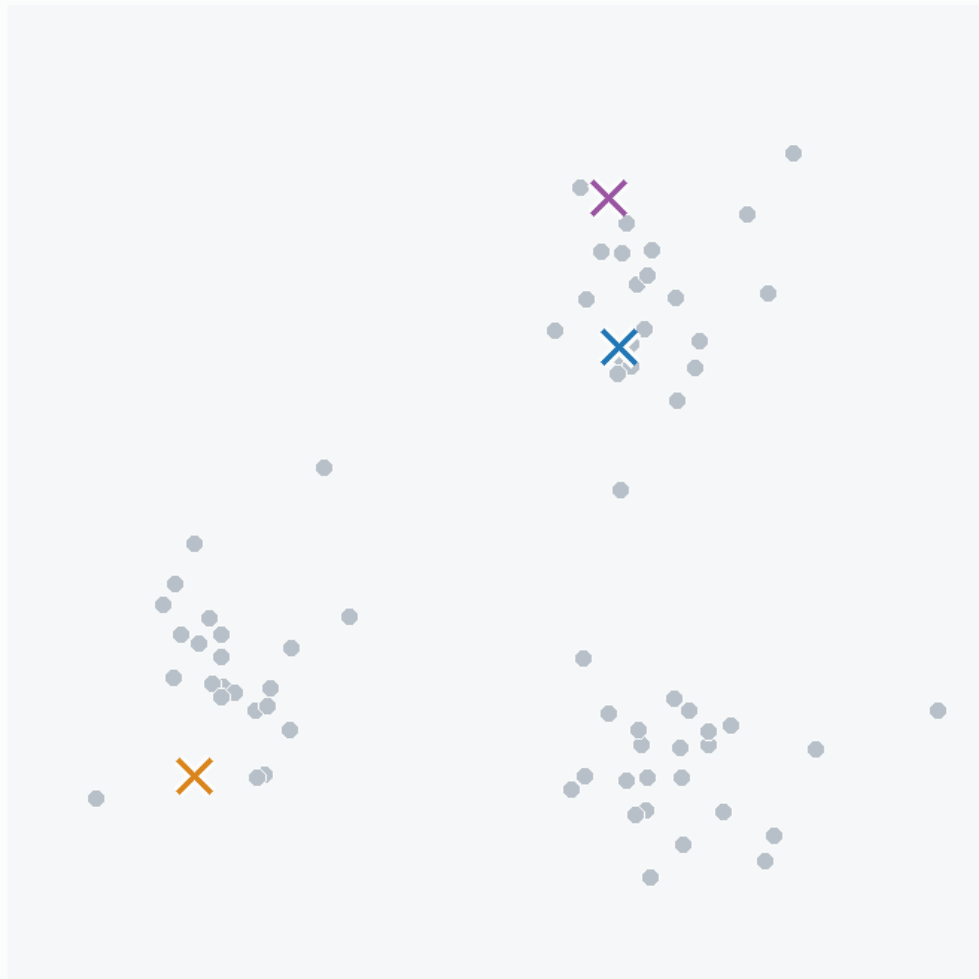
1. Given K , randomly initialize the cluster centers, $c_1, \dots, c_K$
2. Given cluster centers, determine points in each cluster
 - For each point p_i , find the closest c_j . Put p_i into cluster j
3. Given points in each cluster, solve for c_j
 - Set c_j to be the mean of points in cluster j
4. If c_j have changed (up to some threshold), repeat Steps 2, 3.



k-means clustering

$k = 3$

Feature 2



Feature 1

Initialize

Choose three data points as the starting centres.

1 Initialize centres

2 Assign points

3 Update centres

Repeat assignment and update until the centres stop changing.

Small dots: data points X: centres

The Math of k-Means Clustering

Input: Given a set of data points P , number of clusters k

1. Randomly pick k points from P as the centers (means) $c_j, j = 1, \dots, k$
2. Iterate (until max-iterations or c_j no longer changes up to some threshold)
 - Assign each point to nearest center: $\underbrace{y_i}_{\text{cluster id of point } i} = \arg \min_j \|p_i - c_j\|^2$
 - Re-estimate each center as mean of points assigned to it: $c_j = \frac{\sum_{i: y_i=j} p_i}{\sum_{i: y_i=j} 1}$
points i belonging to cluster j

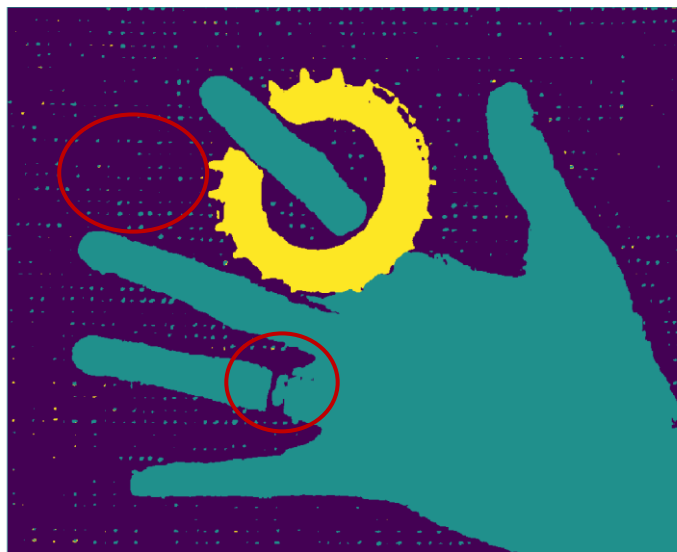
- Minimization of an objective function: $\min_{c, y} \sum_i \|p_i - c_{y_i}\|^2$
- Every iteration of k-means takes a downward step
 - Fixes c and sets y to minimize objective;
 - Fixes y and sets c to minimize objective
- Always converges, but likely to a local minimum

K-Means on Images

$K=2$



$K=3$



Essentially a non-uniform quantization of the image intensities.

Note the spatial discontinuities!

Feature Selection for Clustering

Depending on what we choose as the *feature space*, we can group pixels in different ways.

Grouping pixels based on **intensity** similarity.



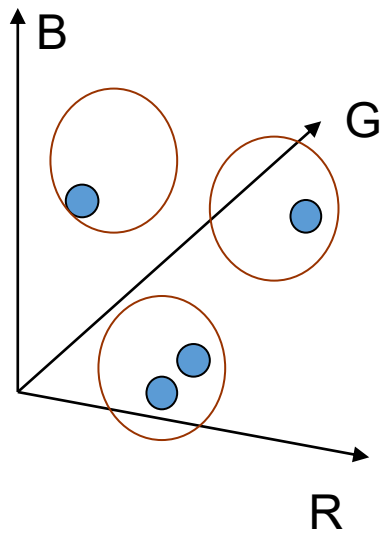
Clusters based on intensity don't have to be spatially coherent.



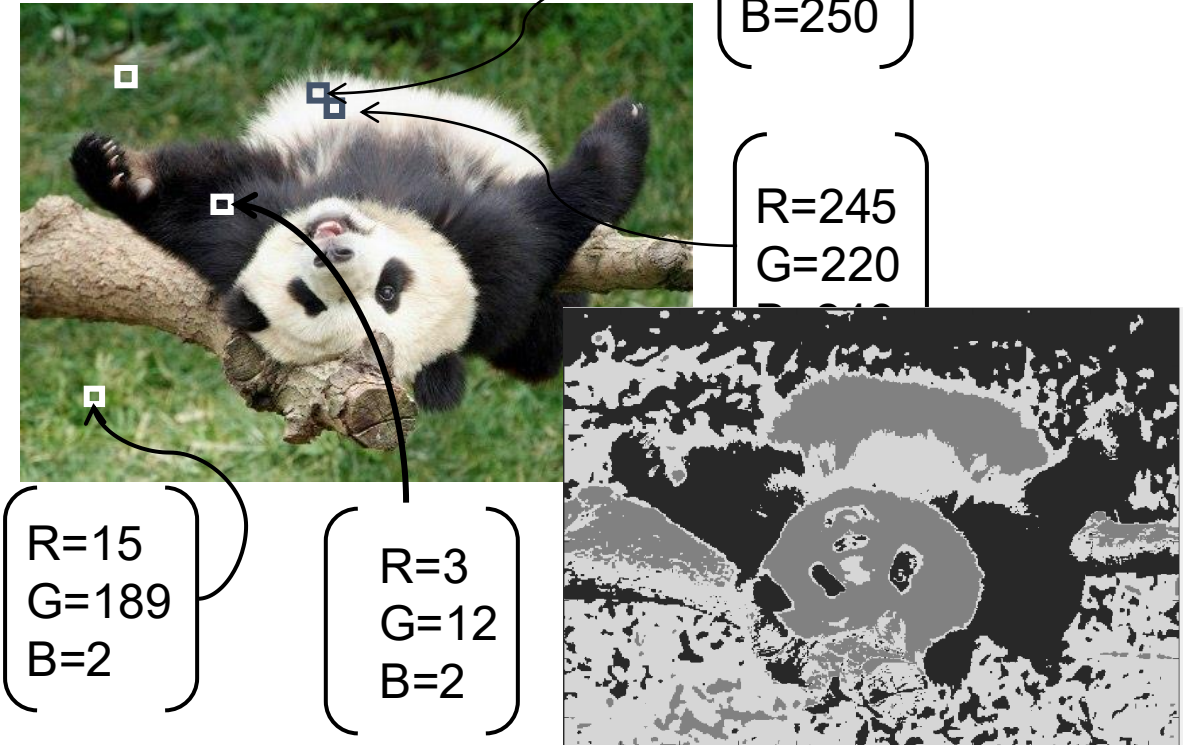
Feature Selection for Clustering

Depending on what we choose as the *feature space*, we can group pixels in different ways.

Grouping pixels based on **color** similarity



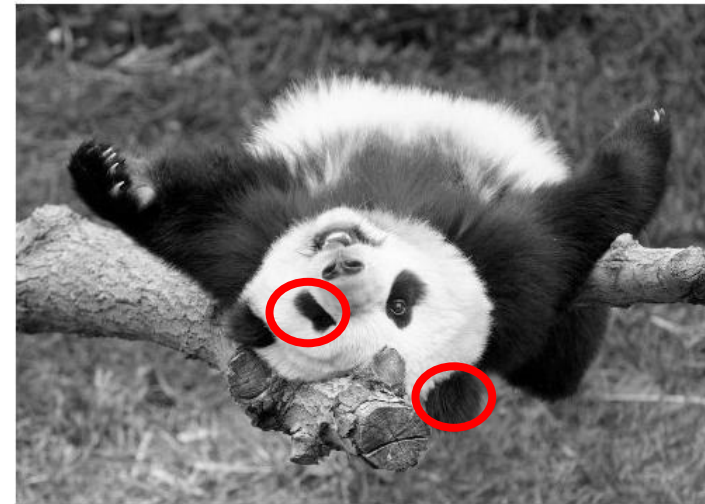
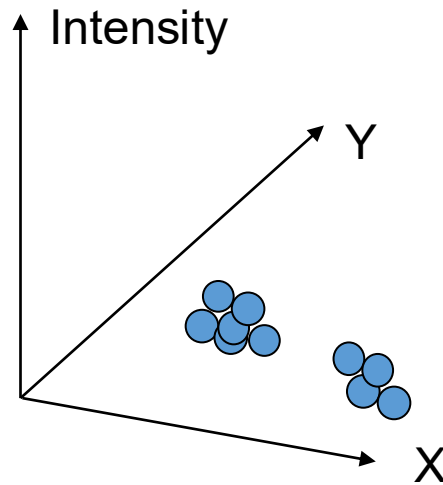
Feature space: color value (3-d)



Feature Selection for Clustering

Depending on what we choose as the *feature space*, we can group pixels in different ways.

Grouping pixels based on
intensity+position similarity



Both regions are black, but if we also include **position** (x,y), then we could group the two into distinct segments; way to encode both similarity & proximity.

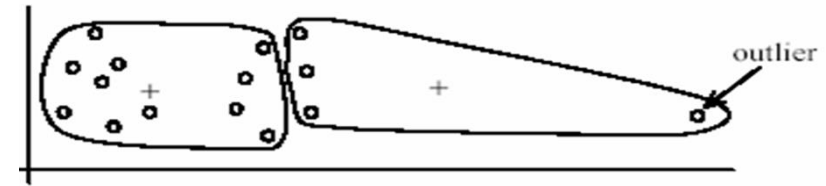
k-Means Clustering: Pros & Cons

Pros

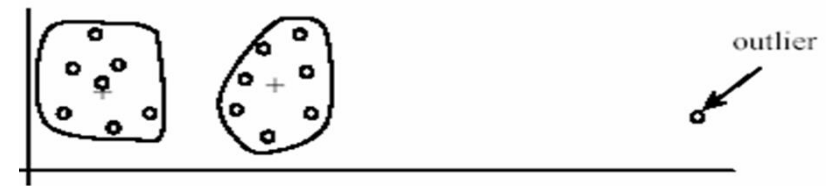
- Simple, fast to compute
- Converges to local minimum of within-cluster squared error

Cons/issues

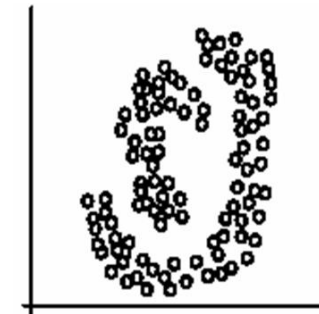
- Setting k ?
- Sensitive to initial centers
- Sensitive to outliers
- Detects spherical clusters
- Assumes means can be computed (efficient, meaningful)



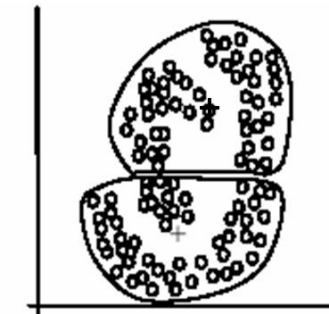
(A): Undesirable clusters



(B): Ideal clusters



(A): Two natural clusters



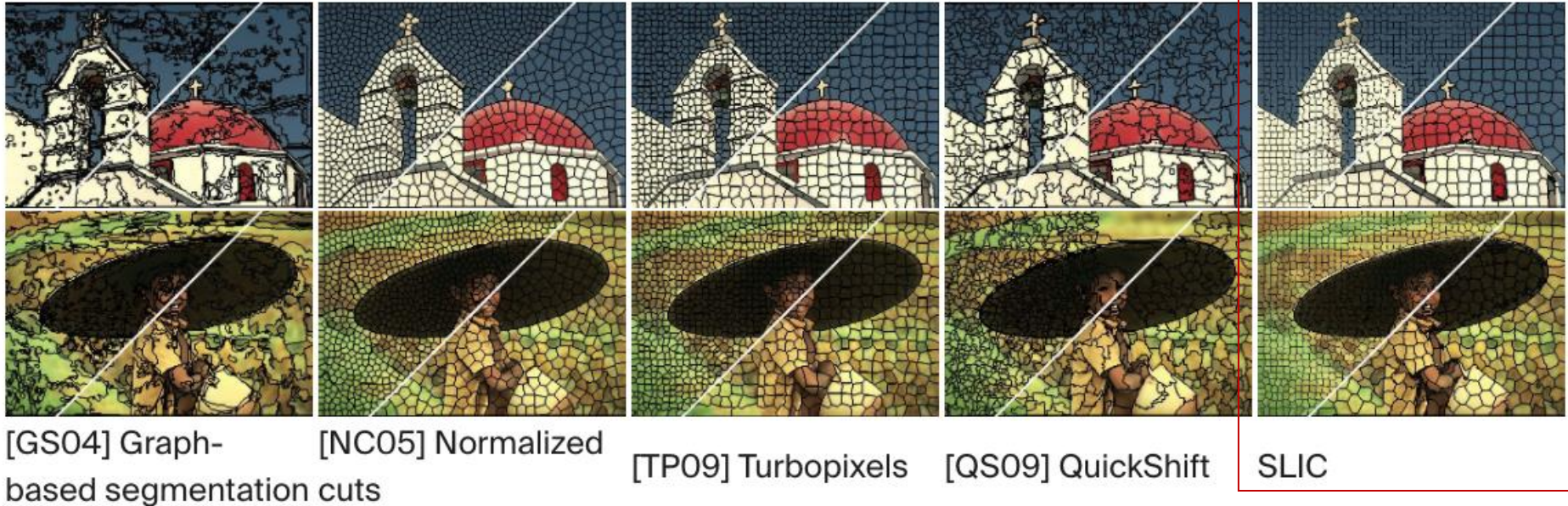
(B): k -means clusters

SLIC Superpixels

Simple Linear Iterative Clustering Superpixels

Adapting k-Means Clustering

What is a Superpixel?



- A group of pixels that share common characteristics, e.g. pixel intensity
- Use as inputs to other CV algorithms since it is a convenient and compact representation with perceptual meaning since its member pixels share visual properties
- Some are more regular in shape, others are less; number of superpixels is hyperparameter

Definitions, Features & Distance Measure

$n_{tp}; n_{sp}$ Total number of pixels in image; number of superpixels we want

$s = [n_{tp} / n_{sp}]^{1/2}$ Initial spacing of each superpixel

Features $z = [r, g, b, x, y]$

$\underbrace{\hspace{1.5cm}}$
colour
 $\underbrace{\hspace{1.5cm}}$
position

Colour Distance $d_c = [(r_j - r_i)^2 + (g_j - g_i)^2 + (b_j - b_i)^2]^{1/2}$

Grey-scale version
 $d_c = [(l_j - l_i)^2]^{1/2}$

L2 distance measures

Spatial Distance $d_s = [(x_j - x_i)^2 + (y_j - y_i)^2]^{1/2}$

Composite Distance $D = \left[\left(\frac{d_c}{d_{cm}} \right)^2 + \left(\frac{d_s}{d_{sm}} \right)^2 \right]^{1/2} \Rightarrow D = \left[d_c^2 + \left(\frac{d_s}{s} \right)^2 c^2 \right]^{1/2}$

scaling factors;
set as max expected values of d_c and d_s

Consider as hyperparameter weighing
relative importance between colour
similarity vs. spatial proximity.

$$d_{sm} = s = [n_{tp} / n_{sp}]^{1/2}$$

Initialization

1. Initialize the algorithm: Compute the initial superpixel cluster centers,

$$\mathbf{m}_i = [r_i \ g_i \ b_i \ x_i \ y_i]^T, \ i = 1, 2, \dots, n_{sp}$$

by sampling the image at regular grid steps, s . Move the cluster centers to the lowest gradient position in a 3×3 neighborhood. For each pixel location, p , in the image, set a label $L(p) = -1$ and a distance $d(p) = \infty$.

Different from k-means; rather than initialize randomly, initialize cluster centers on a grid.

Do not initialize cluster center on an edge or noise point (these have some more extreme values); convergence is faster if we can already initialize on a value common to the surrounding pixels, (approximate as lowest gradient position).



Update + Convergence

2. Assign samples to cluster centers: For each cluster center $\mathbf{m}_i$, $i = 1, 2, \dots, n_{sp}$, compute the distance, $D_i(p)$ between $\mathbf{m}_i$ and *each* pixel p in a $2s \times 2s$ neighborhood about $\mathbf{m}_i$. Then, for each p and $i = 1, 2, \dots, n_{sp}$, if $D_i < d(p)$, let $d(p) = D_i$ and $L(p) = i$.

Compute distance only to closest set of cluster centers.

3. Update the cluster centers: Let C_i denote the set of pixels in the image with label $L(p) = i$. Update $\mathbf{m}_i$:

Same as k-means

$$\mathbf{m}_i = \frac{1}{|C_i|} \sum_{\mathbf{z} \in C_i} \mathbf{z} \quad i = 1, 2, \dots, n_{sp}$$

where $|C_i|$ is the number of pixels in set C_i ,

4. Test for convergence: Compute the Euclidean norms of the differences between the mean vectors in the current and previous steps. Compute the residual error, E , as the sum of the n_{sp} norms. If $E < T$, where T a specified nonnegative threshold, go to Step 5. Else, go back to Step 2.

Same as k-means

Post-Processing

5. *Post-process the superpixel regions:* Replace all the superpixels in each region, C_i , by their average value, $\mathbf{m}_i$.

Optional, to create the “stained-glass” effect.



Example

$$W = 512, \quad H = 384,$$

$$n_{sp} = 100$$

$$n_{tp} = 512 \times 384 = 196608$$

$$s = (196608/100)^{0.5} \approx 44$$

$$\frac{512}{44} \approx 12, \quad \frac{384}{44} \approx 9$$

1. Initialize superpixels as grid of 9×12 .
2. For each cluster center, check distance to all pixels within 88×88 range; assign pixels to closest (checked) center.
- 3./4. Update cluster centers, repeat assignment loop until convergence, i.e. no change up to some threshold.
5. Post-process.



Extra Notes

- At its core, is a modification of k -means clustering
- No provision made to enforce connectivity of superpixels, so it is possible for isolated pixels to remain after convergence. Can do some post-processing e.g. with connected components to re-assign
- Method equally applicable to other color spaces, e.g. HSV. Vector $\mathbf{z}$ can actually be defined as any real-valued feature, as long as we can define a meaningful distance measure
- A superpixel is not an object or semantic category, but a small region intended to preserve likely boundaries.

Semantic Segmentation w/ Neural Networks

Deconvolution, Encoder/Decoders
Fully Convolutional Networks (FCNs), UNet
Mask R-CNN for Instance Segmentation

Segmentation as Dense Classification

Segmentation performs a classification on every pixel in the image

Input RGB image:

$H \times W \times 3$

H,W are image dimensions

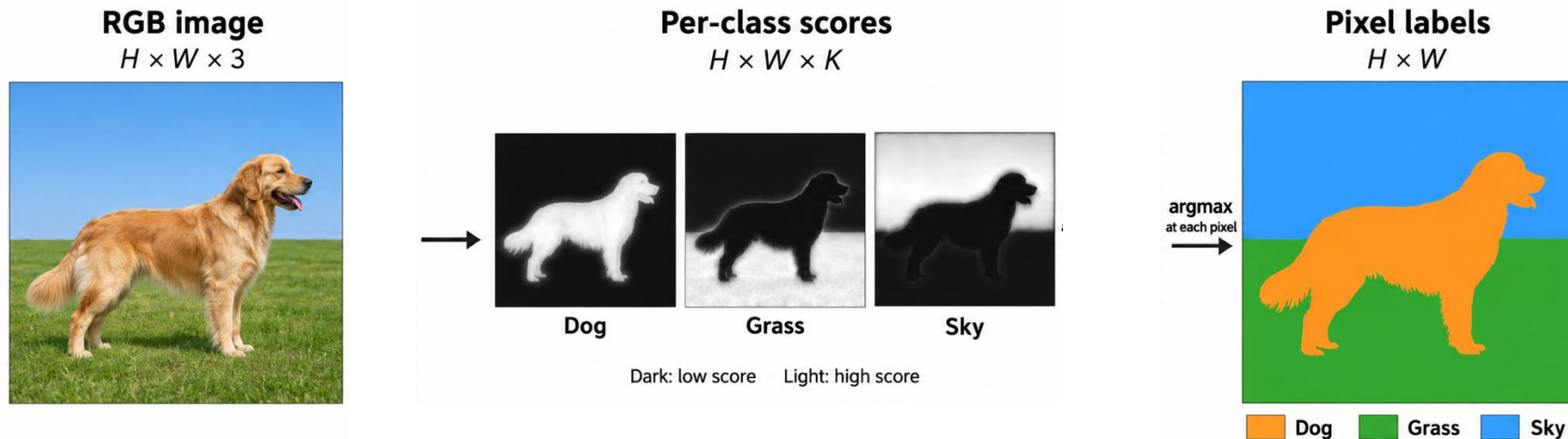
Output as a per-class score map:

$H \times W \times K$

K is the total # of segmentation classes

Flattened Prediction:

$H \times W$



Segmentation as Dense Classification

- Segmentation performs a classification on every pixel in the image
- Label each pixel with a class
 - Training label: a class (index) per pixel
- Prediction output
 - For each pixel, a probability vector (one per class)



Training label:

(0, 0) bicycle ...

(200, 80) people (200, 81) people

Prediction output:

(0, 0) background 0.9; mountain: 0.1 ...

(200, 80) people 0.8; bicycle 0.1; tree: 0.1

The term semantic implies that the segmentation class is linked to the underlying semantics of the object / class to which the pixels belong.

(Semantic) Segmentation Applications

Medical imaging



Delineate organs for treatment planning

Self-driving cars



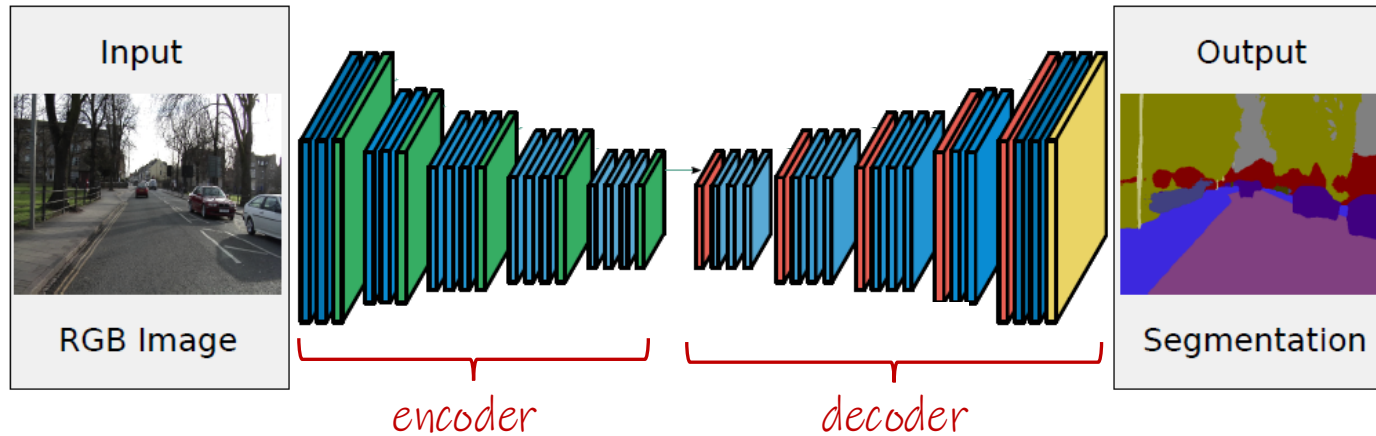
Identify roads, vehicles and pedestrians

Satellite mapping



Map buildings, water and vegetation

Image Encoding / Decoding



Encoder: “read” inputs into (hidden) feature representations h

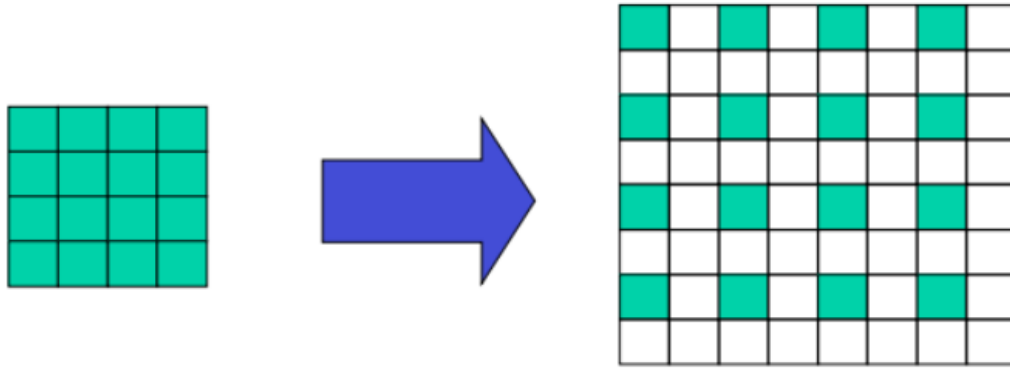
- extracts a semantic-rich representation which can be used directly for label prediction
- subsampling by (pooling or convolution with stride > 1)

Decoder: “write” outputs conditioned on h

- incorporates location information for a learned form of up-sampling
- generates a final feature map as the same size as the input

Original image:  x 10

Upsampling



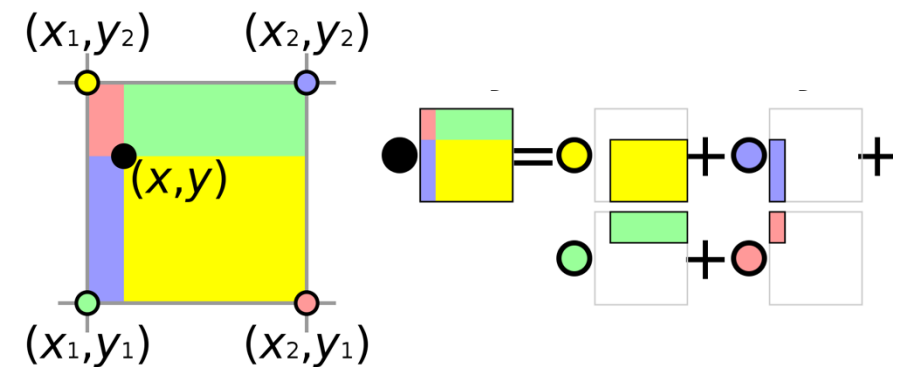
- How to fill in the unknown pixels?
- Nearest-neighbor $\rightarrow$ blocky artifacts
- Bi-linear interpolation / filtering creates a smoother output
- Learn an up-sampling



Nearest-neighbor interpolation



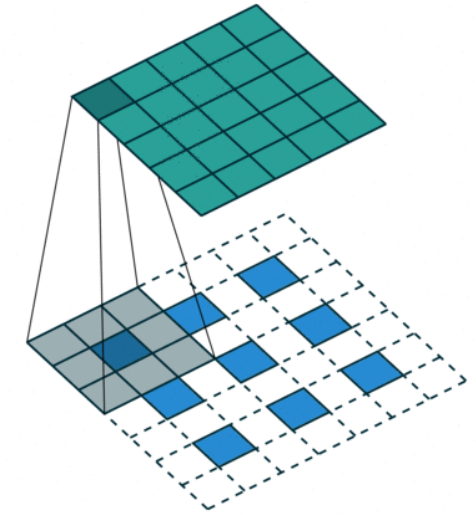
Bilinear interpolation



By Cmglee - Own work, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=21409164>

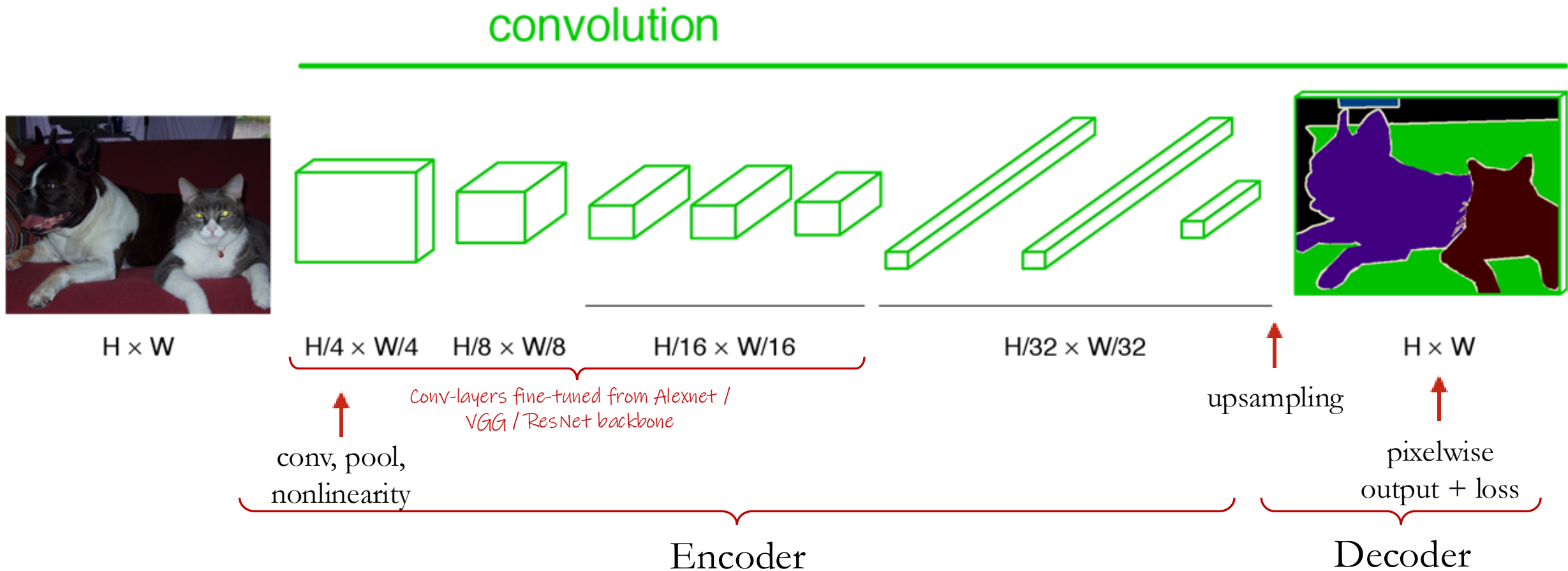
“de”-Convolution

- Up-sampling is actually a convolution
 - Consider that we have “same” padding
 - $\text{stride} = 1 \rightarrow$ output size is same size as input
 - $\text{stride} > 1 \rightarrow$ output size smaller than input
 - $\text{stride} < 1 \rightarrow$ output size bigger than input!
 - a $\text{stride} < 1$ is a fractionally strided convolution
 - equivalent to inserting zeros between elements
 - Stride of $1/s$ is equal to inserting $s-1$ zeros between samples
- Name “transposed convolution” due to implementation
 - (see [link](#)) for detailed numerical explanation and visualization)

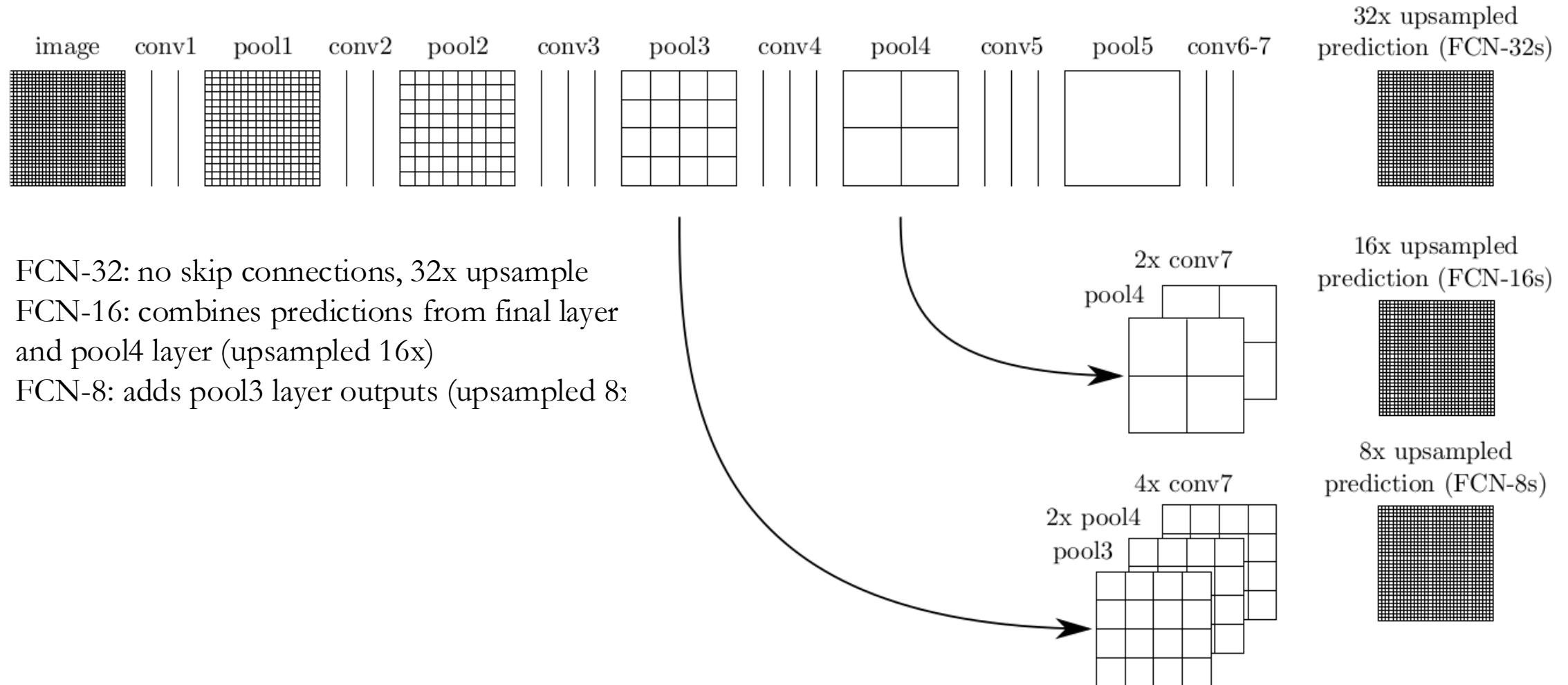


https://github.com/vdumoulin/conv_arithmetic

Fully Convolutional Networks (FCN) [[pdf](#)]



Skip Layers for Refinement



Skip Layers for Refinement

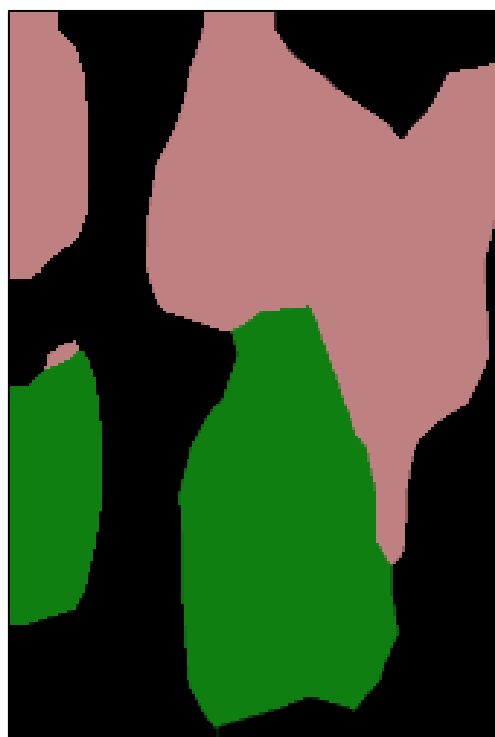
input

stride 32

stride 16

stride 8

truth



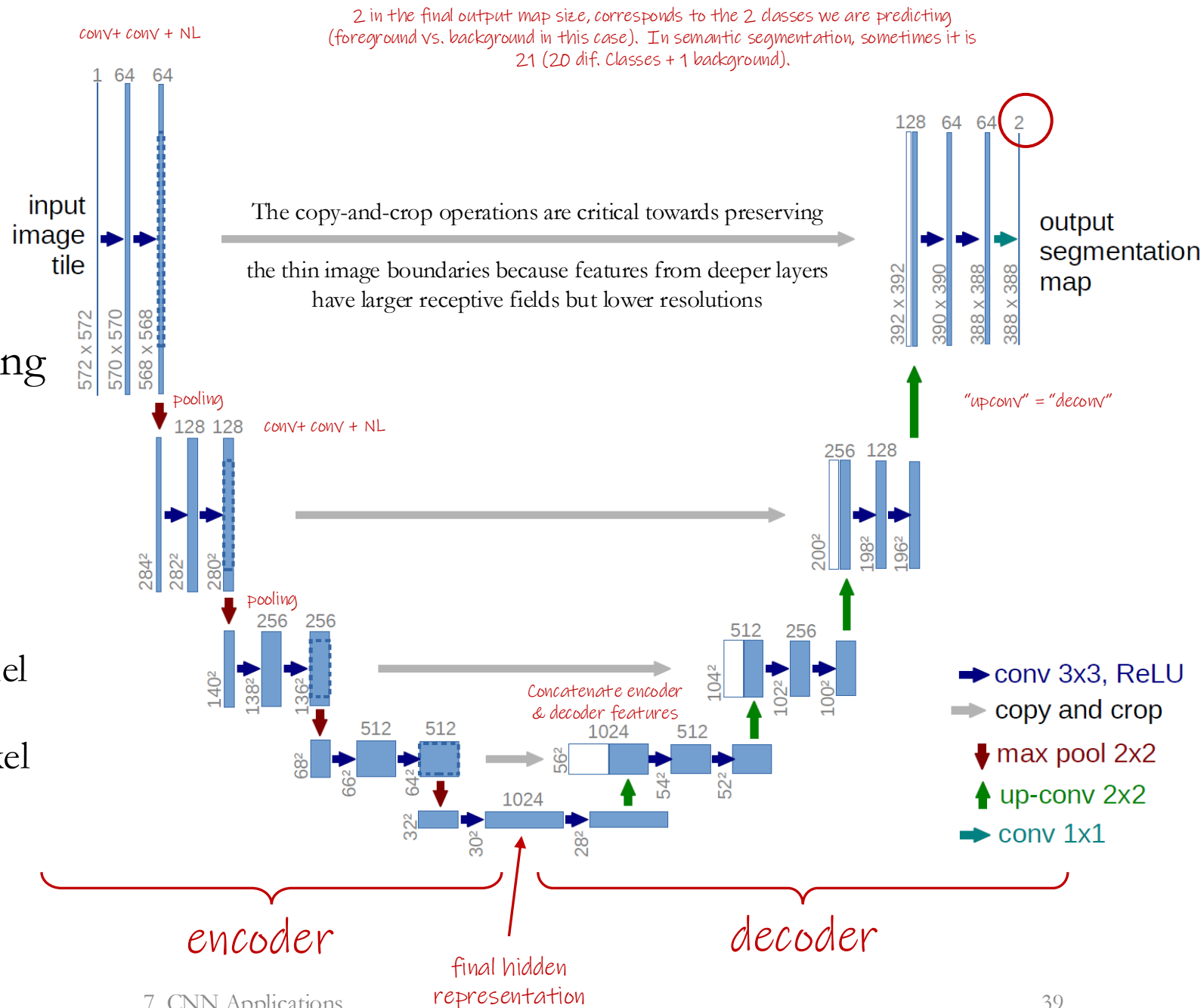
no skips

1 skip

2 skips

U-Net[5]

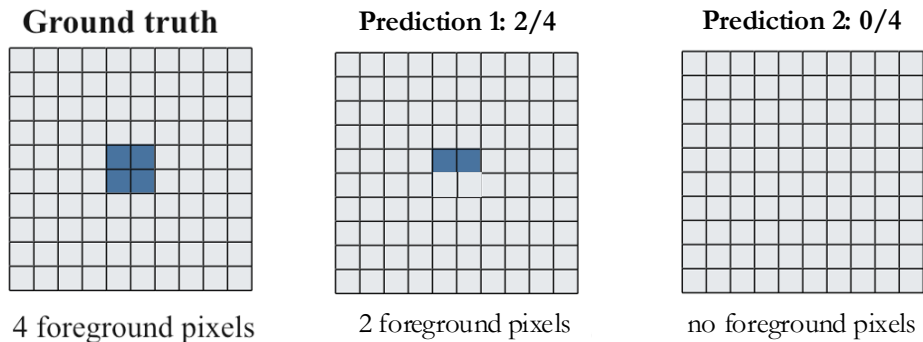
- 3x3 conv + NL + pooling
- Decoder side is smaller than encoder due to “valid” padding
- Final convolution
 - # filters/channels = # classes
- Predict each output pixel independently:
 - Softmax applied on the channel dimension, leading to a probability vector for each pixel
- Cross-entropy computed for each pixel



Evaluation

- Pixel-wise accuracy is a poor measure, if the foreground is only a small %

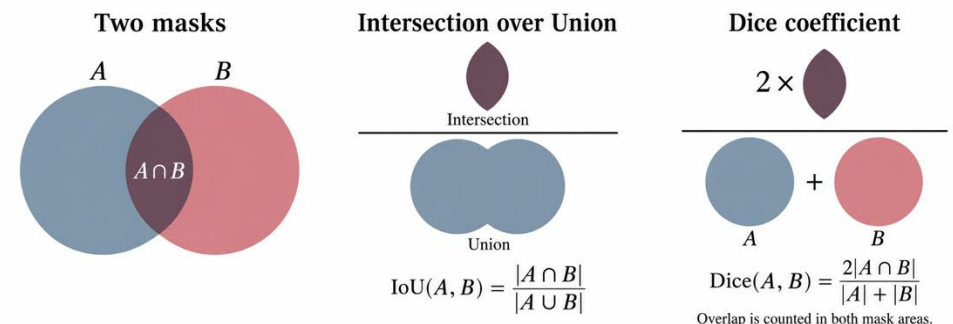
A target occupies 4 of 100 pixels. Predicting only background still gives 96% accuracy.



Prediction	Accuracy	Dice	IoU
All background	96%	0	0
2 of 4 found, no false positives	98%	0.67	0.50
Perfect mask	100%	1	1

- Overlap measures are better at preventing the background from dominating

- Standard IoU:
$$IoU(A, B) = \frac{|A \cap B|}{|A \cup B|}$$
- Dice** coefficient:
$$Dice(A, B) = \frac{2|A \cap B|}{|A| + |B|}$$



** Dice and IoU rank predictions identically, but gives larger numerical scores than IoU. Their distinction comes in the use of Dice (in the soft version) as a loss, because the alternative pixel-averaged cross-entropy loss receives much of its contributions from many background pixels.

Training a Segmentation Network

1. Pixel-wise cross-entropy

Predict the correct class at every pixel.

$$L_{\text{pixel}} = -\frac{1}{N} \sum_{n=1}^N \log p_n(y_n)$$

$p_n(y_n)$: predicted probability of the true class at pixel n

2. Overlap-based losses**

Encourage overlap with the labelled foreground.

Useful when the foreground occupies a small region.

$$L_{\text{overlap}} = 1 - \text{soft overlap score}$$

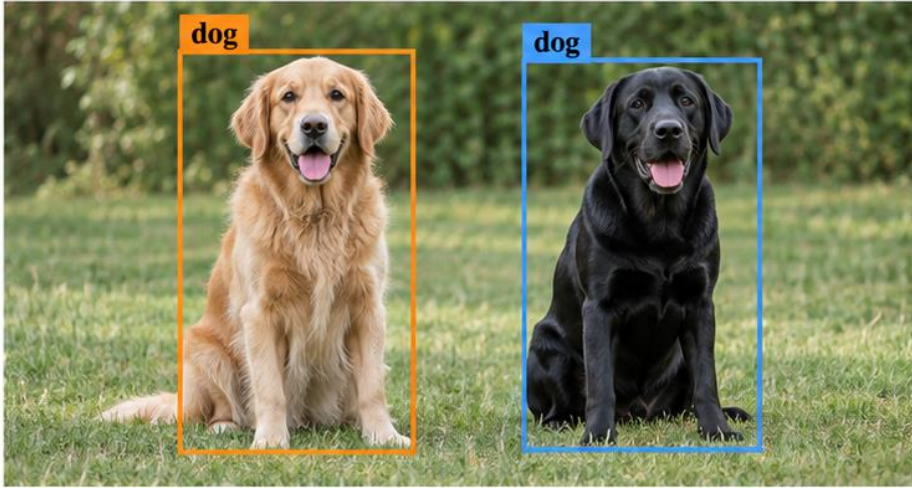
Examples: soft Dice or soft IoU

Can be combined with cross-entropy.

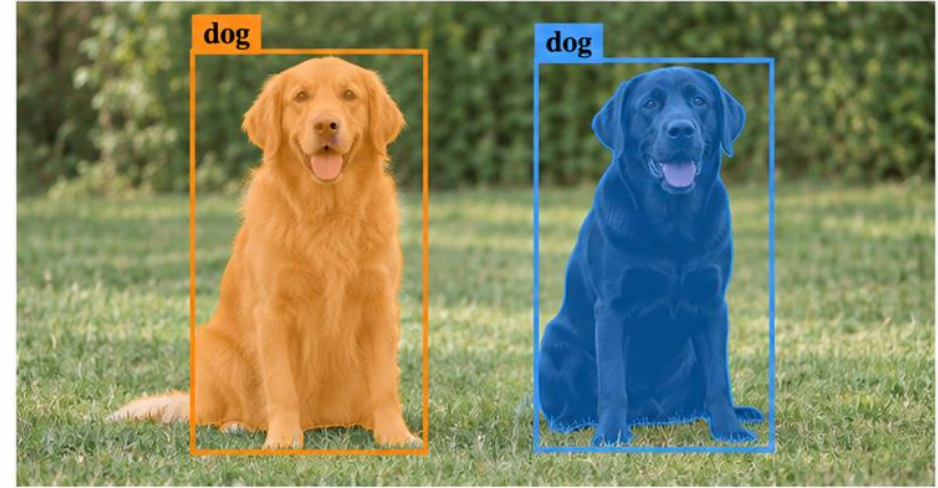
** Exact mechanics of overlap loss beyond our scope. Takeaway is that they use the predicted class probabilities to encourage the predicted region to overlap with the ground truth.

From Detection to Instance Segmentation

Object detection

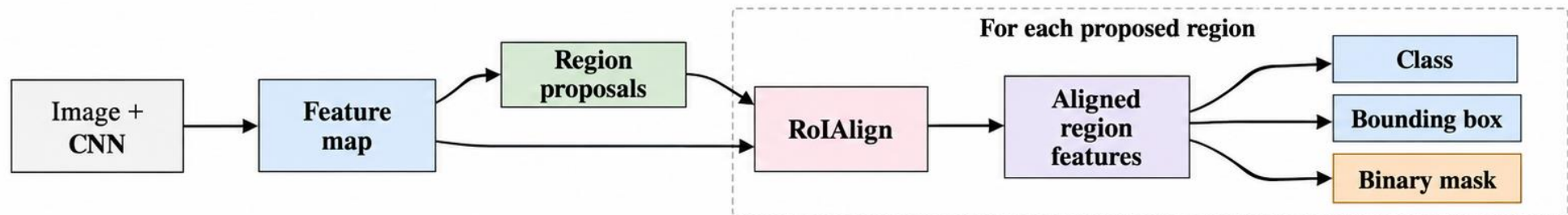


Instance segmentation



Illustrative outputs

Same class, separate instances.

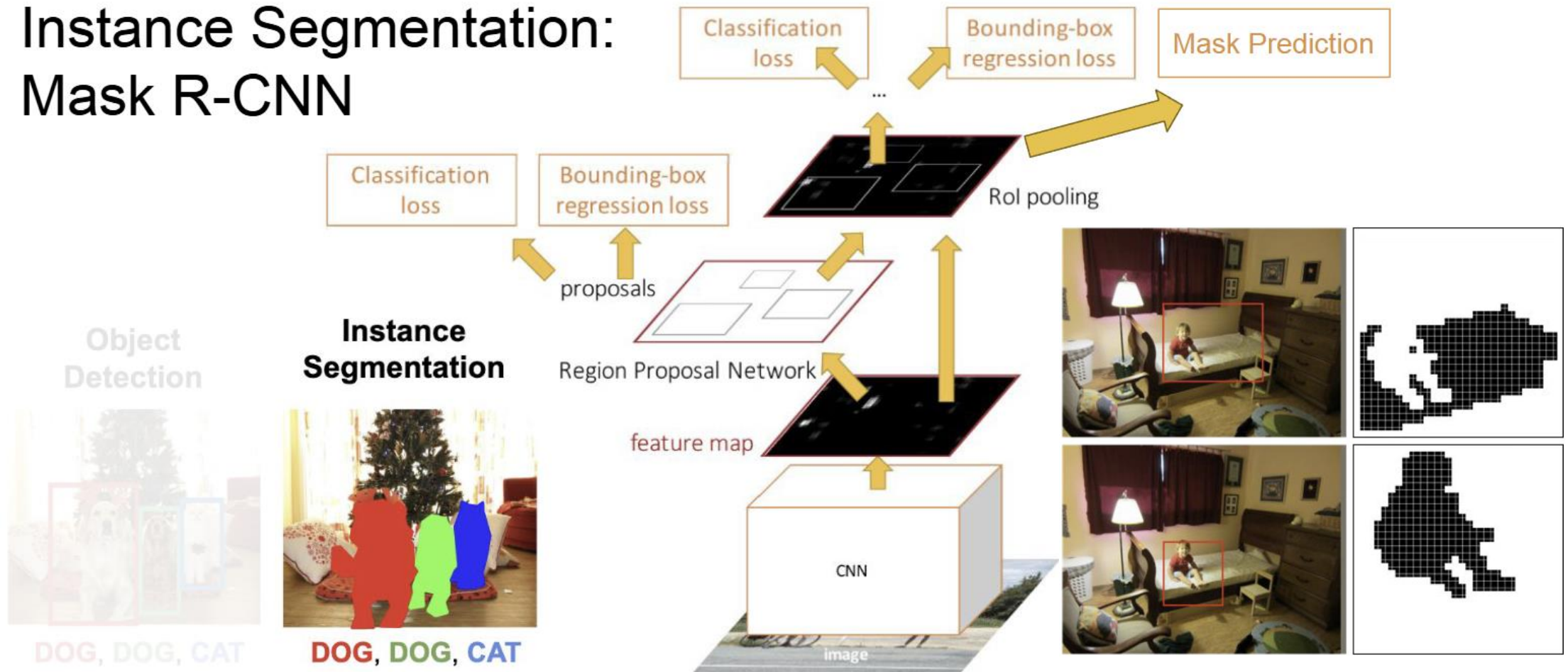


The mask branch predicts which pixels belong to each detected object.

RoIAlign preserves spatial alignment when extracting region features.

Mask R-CNN Adds a Mask Prediction Head

Instance Segmentation: Mask R-CNN



Promptable Segmentation

Segment Anything (SAM), SAM2, SAM3 ...

Can We Specify What to Segment?

- A **segmentation prompt** is an additional input specifying which object or region to segment
- The prompt guides the segmentation, but does not retrain the model itself

Points



Include here (+); exclude here (-)

Box



Specify the approximate extent

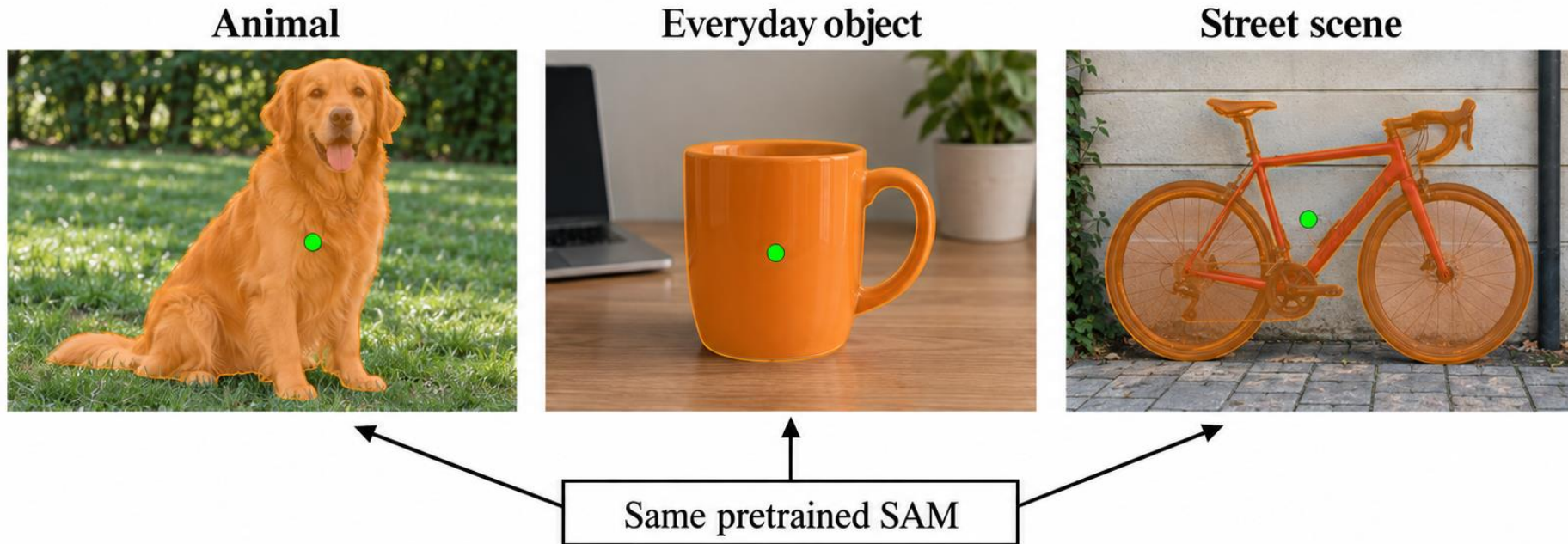
Mask



Provide or refine a rough mask

A Reusable Segmentation Model

Pretrain once, then prompt on new images.



New image + prompt: predict a mask without retraining for each image.

Training teaches the model; the prompt specifies the selection at inference.

Transfer is not guaranteed: unfamiliar domains and fine boundaries can remain difficult.

Illustrative examples

Segment Anything Model (SAM)

Segment Anything

Alexander Kirillov^{1,2,4} Eric Mintun² Nikhila Ravi^{1,2} Hanzi Mao² Chloe Rolland³ Laura Gustafson³
Tete Xiao³ Spencer Whitehead Alexander C. Berg Wan-Yen Lo Piotr Dollár⁴ Ross Girshick⁴
¹project lead ²joint first author ³equal contribution ⁴directional lead

Meta AI Research, FAIR

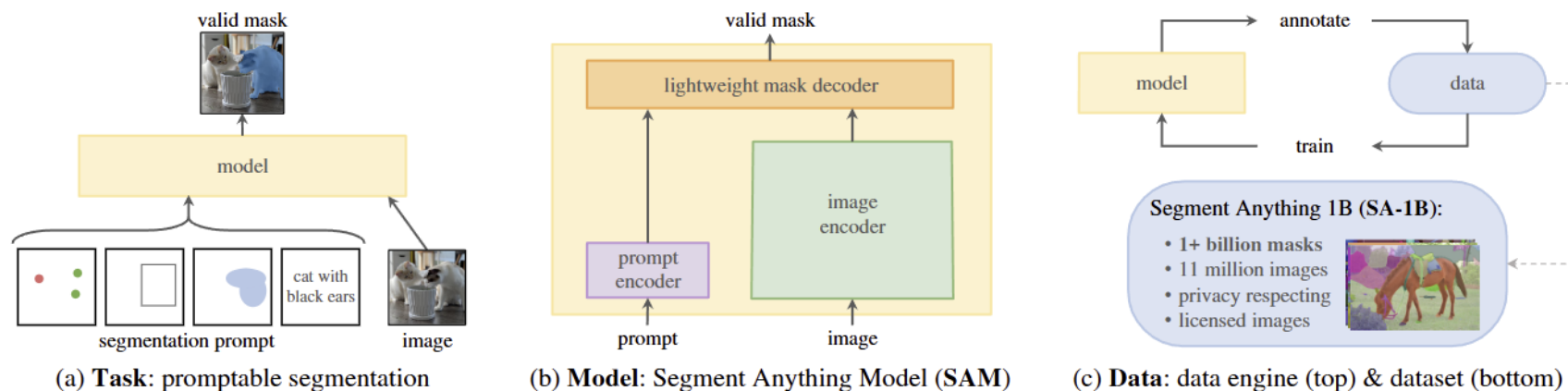
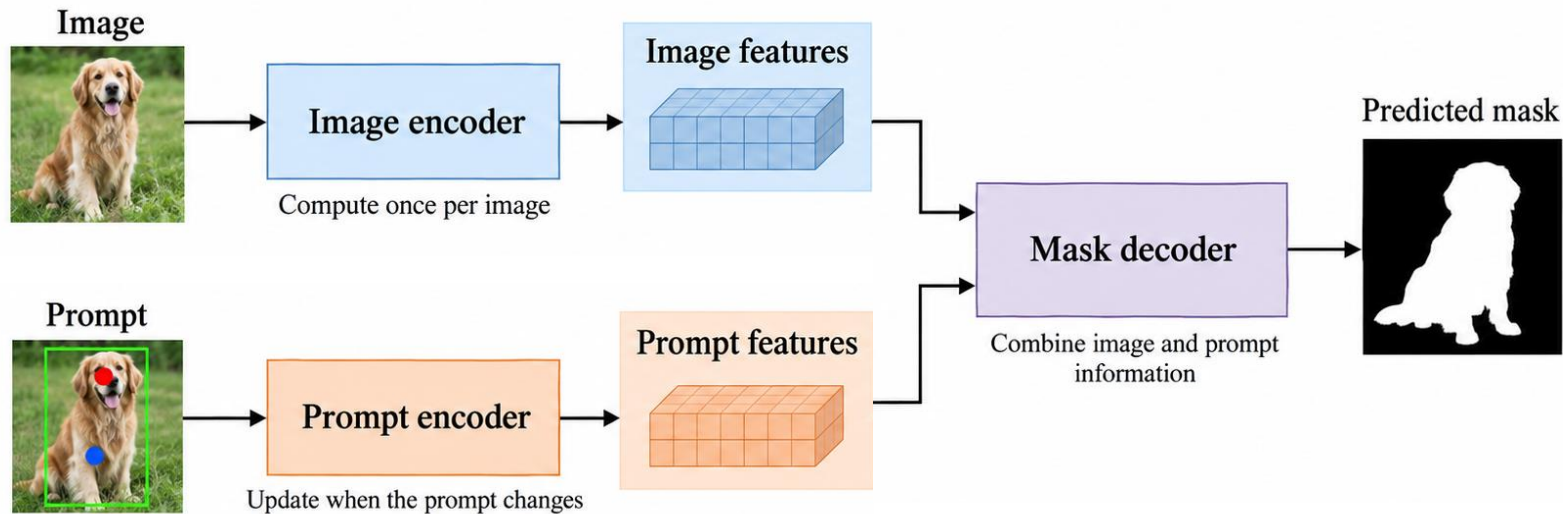


Figure 1: We aim to build a foundation model for segmentation by introducing three interconnected components: a promptable segmentation *task*, a segmentation *model* (SAM) that powers data annotation and enables zero-shot transfer to a range of tasks via prompt engineering, and a *data* engine for collecting SA-1B, our dataset of over 1 billion masks.

How SAM Produces a Segmentation Mask

- SAM: Segment Anything foundation model
- The same image feature supports many different prompts
- The decoder** predicts a mask matching the prompt



** Encoder and decoder internals are beyond this lecture's scope.

Prompts May Be Ambiguous

- A single click may identify a part or a whole object
- Positive & negative clicks for including and excluding regions.



SAM can return several candidate masks for an ambiguous prompt.
Positive clicks include regions; negative clicks help exclude unwanted regions.

SAM 2: Segmentation in Videos

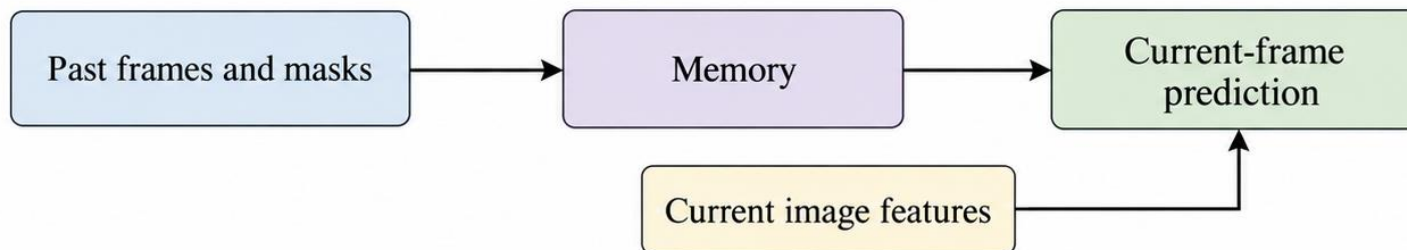
Keep track of the selected object as the scene changes.



Frame 1: select

Frame 2: propagate

Frame 3: continue



Memory carries information about the target across frames.

Add corrective prompts when the selection drifts or becomes uncertain.

SAM 2 supports both image and video segmentation.

Illustrative video sequence

SAM 3: Segmentation With Concepts

Specify a concept and find all matching instances.

“yellow school bus”



Text prompt + image

An exemplar prompt can also specify the concept by showing an example.

SAM 3 can detect, segment and track matching objects in images and videos.

Spatial prompt: this object. Concept prompt: objects matching this description.

Illustrative example

What Changes Across Segmentation Systems?

System	What specifies the target?	Typical output	Main idea
U-Net / FCN	Classes defined during training	A class label at each pixel	Dense classification
Mask R-CNN	Detected object proposals	A class, box and mask per instance	Detection + masks
SAM	Spatial prompts	Selected-region masks	Promptable images
SAM 2	Spatial prompts + video context	Masks across frames	Memory for video
SAM 3	Text or exemplar prompts	Masks for matching instances	Promptable concepts

SAM 2 also handles images; SAM 3 also supports spatial prompts and video.

Which system would you use to follow a clicked object through a video?

What changes if the request is “find every yellow school bus”?

Choose the task and interaction first, then the model.

Summary

Clustering in Feature Space

- Segmentation groups pixels according to similarity in intensity, colour, texture or position.
- K-means alternates between assigning pixels to clusters and updating cluster centres. The features and their scaling determine the resulting groups.

Superpixels

- Superpixels group nearby pixels into compact regions with similar appearance.
- SLIC adapts k-means w/ a distance balancing colour similarity & spatial proximity.
- Superpixels need not correspond to whole objects.

Segmentation with Neural Networks

- FCN and U-Net learn dense pixel predictions using encoders, upsampling and skip connections.
- Mask R-CNN predicts a separate mask for each detected instance.
- Cross-entropy supervises pixel labels, while overlap losses supervise regions. Dice and IoU evaluate the overlap between predicted and ground-truth masks.

Promptable Segmentation

- A prompt specifies the object or region to segment at inference time, without retraining the model.
- SAM family of segmentation foundation models supports various prompts in images and video

Extra Readings

- [Torralba](#) 30.9 Cluster, k-means
31.1-3 Perceptual grouping, segmentation in feature space
24.8, 24.11 Spatial outputs, encoder-decoders, U-Net
50.5-6 Semantic & instance segmentation
- [Szeliski](#) 5.2.1-2 Clustering, k-means, Gaussian mixtures (optional)
6.4 Segmentation
- [Klette](#) 5.1 Classic segmentation

FAQs

Cluster in Feature Space

Q: Two disconnected red objects receive the same cluster label. Has k-means failed?

A: Not necessarily. If the features contain only colour, those pixels may be close in feature space despite being far apart in the image. The result can minimize the clustering objective while failing to separate object instances.

Q: The clustering error decreases, but the segmentation looks worse. Can both be true?

A: K-means minimizes squared feature distances, not disagreement with human object boundaries. Splitting a textured object or merging similarly coloured objects can improve that objective. The feature representation must reflect the intended grouping.

Q: RGB values change from 0–255 to 0–1, but pixel coordinates remain unchanged. Should the result stay the same?

A: No. Colour differences become much smaller relative to spatial differences, so position has more influence. Rescaling one feature group changes the distance measure unless its weight is adjusted accordingly.

Q: If I shuffle the pixels, should colour-only k-means find different colour clusters?

A: The colour samples are unchanged, so the objective is unchanged. With equivalent initial centres and tie handling, the same colours receive the same assignments. The reconstructed label image is shuffled. Position features would make this a different problem.

Q: Why not choose the K with the smallest clustering error?

A: The best achievable error cannot increase as K grows and can become zero with enough centres. This criterion alone rewards excessive splitting. Choosing K needs a desired granularity or another criterion that penalizes unnecessary complexity.

Q: Two runs produce different segmentations with almost equal error. Which is correct?

A: The objective may support several similar solutions, especially with overlapping groups. A small error difference does not establish which matches the task. Examine stability and meaningful boundaries, rather than treating either run as ground truth.

Superpixels & SLIC

Q: The SLIC distance loses a factor of $1/c$. Why can assignments stay unchanged when the numbers change?

A: For a fixed positive c , every candidate distance is multiplied by the same positive factor, preserving their order. The expressions are proportional, not equal. An absolute distance threshold would need rescaling, and a different factor for each centre would not preserve assignments.

Q: My superpixels look like a regular grid but cut through colour boundaries. What would changing c do?

A: Reducing c weakens spatial compactness relative to colour and lets boundaries follow appearance more closely. This may produce less regular shapes. It cannot recover a boundary that the colour features do not distinguish.

Q: A pixel is spatially closest to one centre. Why let a more distant center compete in a $2S \times 2S$ window?

A: The farther centre may be a better match in colour, giving a smaller combined distance. Overlapping windows also let boundaries depart from the initial grid. Assigning only by spatial proximity would prevent much of this adaptation.

Q: Could moving a SLIC centre to the lowest-gradient pixel make its initialization worse?

A: Yes. Low gradient is a local heuristic, not knowledge of the correct segment. It usually avoids edges, but could move a centre from a thin object into its background. The small search neighbourhood limits the displacement without eliminating this risk.

Q: If the superpixel label map still stores one value per pixel, where is the efficiency gain?

A: The gain comes when later operations use one feature vector or decision per region. The label map preserves pixel membership so results can be mapped back. Superpixels do not automatically reduce storage or eliminate every pixel-level computation.

Q: A superpixel contains both foreground and background. Can a later classifier repair its boundary?

A: If the classifier must assign one label to the whole superpixel, it cannot split that region. The initial grouping limits the best possible boundary. Pixel-level refinement can relax this constraint, while smaller superpixels reduce it at greater processing cost.

FAQs

Network Design and Dense Prediction

Q: An upsampled prediction has the original image dimensions. Why can its boundary still be coarse?

A: Resolution describes the number of output samples, not the detail they contain. Upsampling coarse features creates more samples without necessarily restoring lost boundary information. Higher-resolution features and learned refinement can improve localization.

Q: If a decoder can learn, why does it still need skip connections?

A: Different fine structures can produce similar coarse representations after downsampling. A decoder must then infer which structure was present. Skip connections provide additional spatial evidence, reducing this ambiguity rather than merely increasing the output size.

Q: If the loss is evaluated per pixel, can a distant part of the image change a pixel's prediction?

A: Yes, if it lies within the model's effective context. The loss location does not determine the information used to make the prediction. Convolutions across layers or attention can combine evidence from other locations before the pixel's class scores are computed.

Q: Two transposed-convolution examples use the same stride but produce different output sizes. Must one be wrong?

A: No. Output size also depends on kernel size, padding and any output-padding convention. In the zero-insertion explanation, first determine the expanded grid size, then apply the chosen ordinary-convolution padding rule. Stride alone is insufficient.

Q: Can we remove U-Net's final class layer and use its feature channels as class probabilities?

A: Not in general. Hidden channels represent learned features rather than one channel per class. The output layer maps those features to class scores, and the appropriate activation converts the scores to probabilities. Matching the number of channels by chance is not enough.

Training and Evaluation

Q: Two models produce identical binary masks. Must their training losses be equal?

A: No. Probabilities of 0.51 and 0.99 both pass a 0.5 threshold, but express different confidence. Cross-entropy and soft overlap losses can distinguish them. Hard-mask evaluation discards these probability differences.

Q: The loss falls for several epochs while Dice on the thresholded masks stays constant. Is learning broken?

A: Not necessarily. Probabilities may improve without crossing the threshold, so the hard mask does not change. A soft training objective can therefore improve before the evaluation score does. Persistent disagreement is a reason to inspect predictions, not proof of a bug.

Q: A model reports Dice 0.80 and another reports IoU 0.70. Which is better?

A: The raw scores are not directly comparable. For one binary-mask comparison, Dice 0.80 corresponds to IoU about 0.67. Dataset averages require more care because the conversion is nonlinear. Compare models using the same metric and aggregation protocol.

Q: Would missing ten foreground pixels matter equally for a tiny object and a large one?

A: Not to an overlap metric. Ten missed pixels may be most of a small object but a negligible fraction of a large one. This makes the choice between pooling all pixels and averaging per-object or per-image scores important.

Q: Can adding an overlap loss make every aspect of a segmentation better?

A: No. It changes what training rewards and how errors interact across a region. It may improve overlap without improving boundary precision or probability calibration. The loss should reflect the task, and evaluation should check the properties that matter.

FAQs

Instance Masks and Task Choice

Q: A model correctly labels every dog pixel. Why might it still fail to count the dogs?

A: Its output assigns a shared class label, not a separate identity to each dog. Touching dogs can form one connected region, while one occluded dog may appear disconnected. Counting connected components is therefore not a reliable substitute for instance segmentation.

Q: Could we turn a semantic segmentation model into an instance model by adding more output classes?

A: Giving each possible object a fixed class channel does not solve the variable number and arbitrary ordering of instances. Instance methods need a mechanism for separating individual objects, such as object proposals and a mask prediction for each proposal.

Q: Mask R-CNN predicts a good mask inside a region, but the region belongs to the wrong class. Is the result correct?

A: It may have good geometric overlap but still be an incorrect class-labelled instance prediction. Class correctness, localization and mask quality are related but distinct requirements. A good result in one branch does not guarantee the full detection is correct.

Q: A promptable model gives an excellent mask after I draw a tight box. Is that a fair comparison with automatic instance segmentation?

A: Only if the comparison accounts for the extra input. The box already provides target selection and localization. Automatic instance segmentation must find the instances itself. State what prompts or external detections each system receives.

Q: If a model can segment many kinds of objects, why might we still train a task-specific model?

A: The task may require automatic class labels, consistent outputs on a particular image domain, or a specific computation budget. Broad promptability does not guarantee the best performance for every deployment. Evaluate the complete workflow, including how targets are specified.

Reasoning About Prompts

Q: I click on a person’s sleeve and get a mask of the whole person. Is the prediction necessarily wrong?

A: The click alone may be compatible with the sleeve, shirt or person. Correctness depends on the intended target, which the prompt may not specify fully. More precise prompts or a choice among candidate masks can resolve the ambiguity.

Q: Why can a negative click help even when it is outside the desired object?

A: It identifies a region the mask should exclude. This can distinguish a narrow target from a larger candidate that contains both positive and negative points. The model uses the combination of prompts to refine the intended extent.

Q: Does adding more clicks always improve the mask?

A: No. Contradictory or misplaced clicks can confuse the target specification. Even consistent clicks may not fix an unfamiliar appearance or a difficult boundary. Useful additional prompts supply information that distinguishes the intended region from competing interpretations.

Q: Why is running an image segmentation model independently on each video frame insufficient for tracking?

A: Good individual masks do not establish which region corresponds to the same object over time. Similar objects, occlusion and appearance changes create ambiguity. Video memory supplies temporal context, although it does not guarantee perfect identity preservation.

Q: How does “segment this bus” differ from “find every yellow bus”?

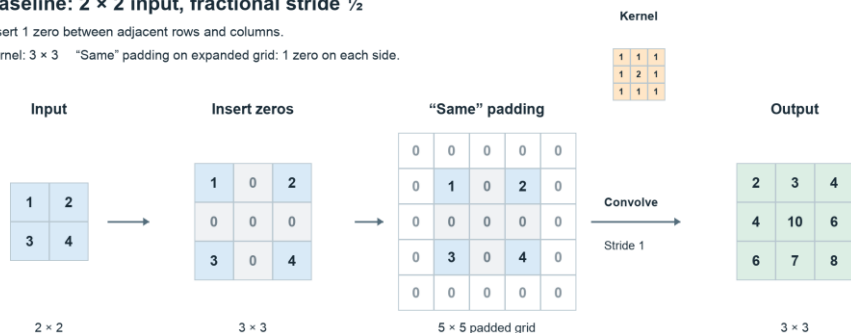
A: A spatial prompt can select a particular bus already localized in the image. The concept query requires discovering all matching instances and deciding whether each satisfies the description. The required output changes from a selected-region mask to a set of matching instances.

Deconvolution Examples

Baseline: 2×2 input, fractional stride $\frac{1}{2}$

Insert 1 zero between adjacent rows and columns.

Kernel: 3×3 "Same" padding on expanded grid: 1 zero on each side.



Output height: $(2 - 1) \times 2 + 1 = 3$ Output width: $(2 - 1) \times 2 + 1 = 3$

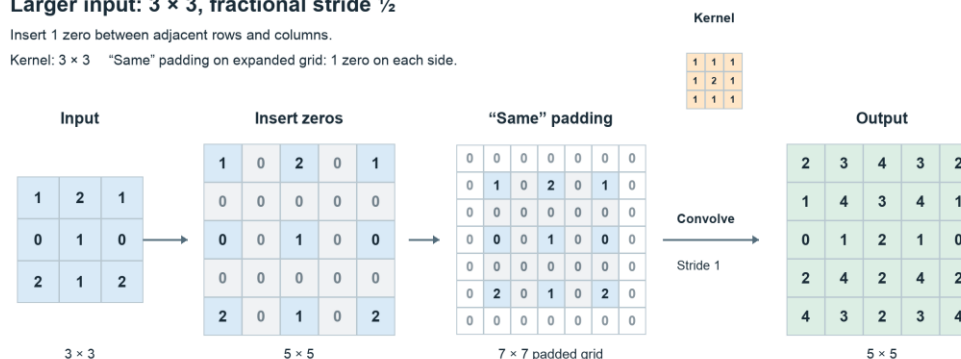
The stride-1 convolution preserves the expanded grid size.

Blue: original entries Grey: inserted zeros White border: padding Orange: kernel

Larger input: 3×3 , fractional stride $\frac{1}{2}$

Insert 1 zero between adjacent rows and columns.

Kernel: 3×3 "Same" padding on expanded grid: 1 zero on each side.



Output height: $(3 - 1) \times 2 + 1 = 5$ Output width: $(3 - 1) \times 2 + 1 = 5$

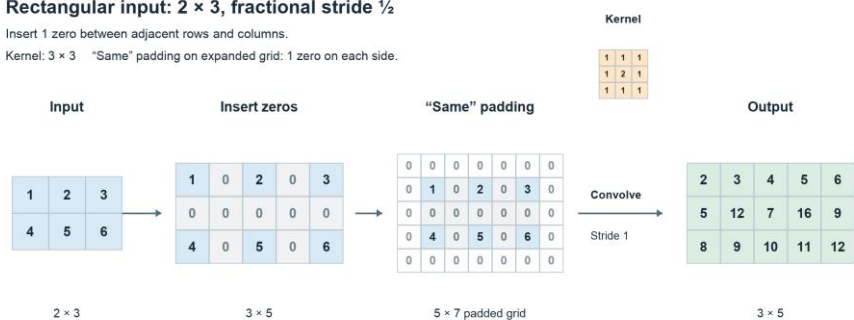
The stride-1 convolution preserves the expanded grid size.

Blue: original entries Grey: inserted zeros White border: padding Orange: kernel

Rectangular input: 2×3 , fractional stride $\frac{1}{2}$

Insert 1 zero between adjacent rows and columns.

Kernel: 3×3 "Same" padding on expanded grid: 1 zero on each side.



Output height: $(2 - 1) \times 2 + 1 = 3$ Output width: $(3 - 1) \times 2 + 1 = 5$

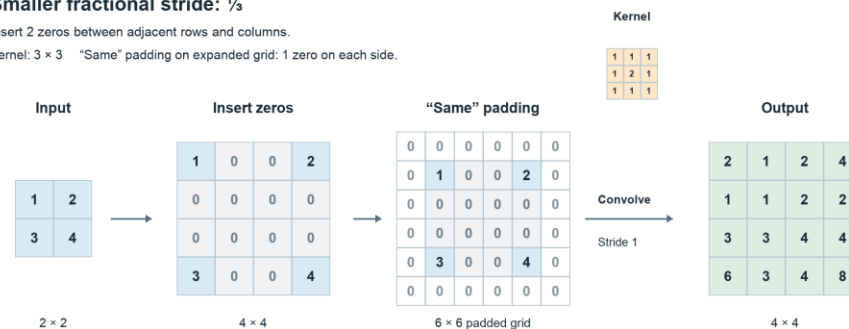
The stride-1 convolution preserves the expanded grid size.

Blue: original entries Grey: inserted zeros White border: padding Orange: kernel

Smaller fractional stride: $\frac{1}{3}$

Insert 2 zeros between adjacent rows and columns.

Kernel: 3×3 "Same" padding on expanded grid: 1 zero on each side.



Output height: $(2 - 1) \times 3 + 1 = 4$ Output width: $(2 - 1) \times 3 + 1 = 4$

More inserted zeros increase the expanded grid from 3×3 to 4×4 .

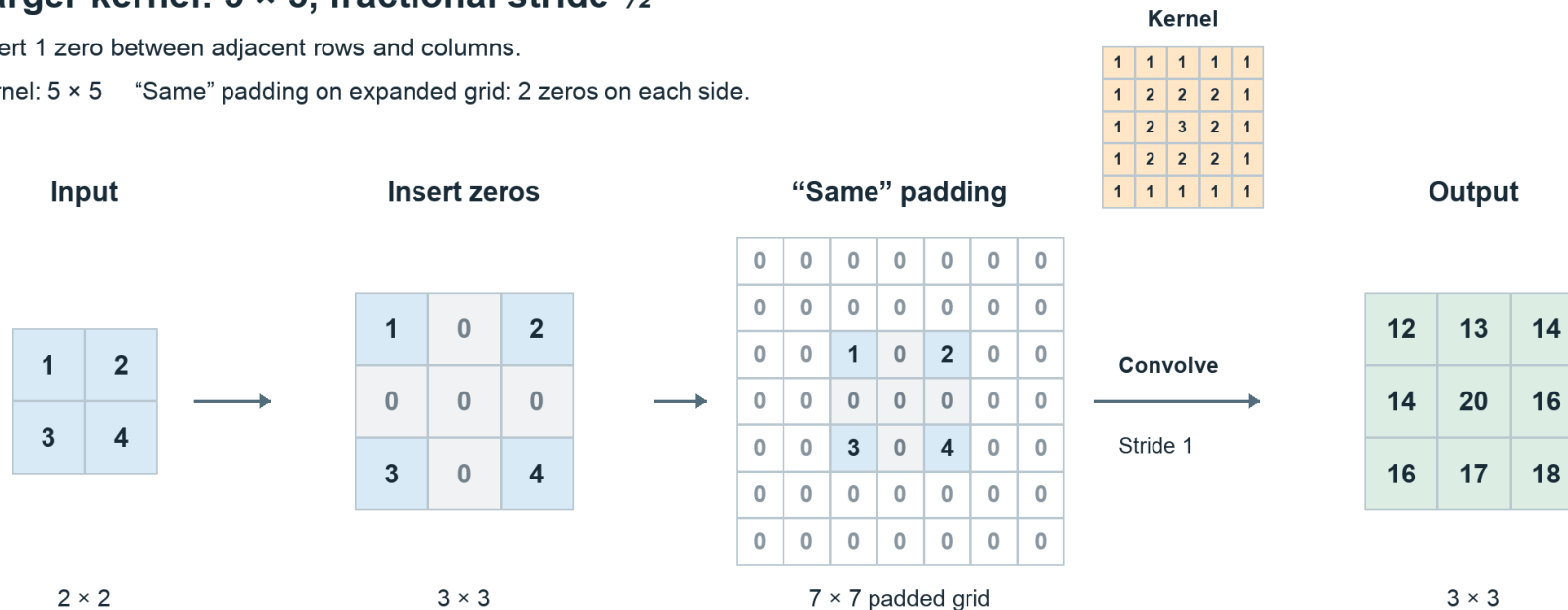
Blue: original entries Grey: inserted zeros White border: padding Orange: kernel

More Deconvolution Examples

Larger kernel: 5×5 , fractional stride $\frac{1}{2}$

Insert 1 zero between adjacent rows and columns.

Kernel: 5×5 "Same" padding on expanded grid: 2 zeros on each side.



Output height: $(2 - 1) \times 2 + 1 = 3$ **Output width:** $(2 - 1) \times 2 + 1 = 3$

A larger kernel changes the values; "same" padding keeps the output at 3×3 .

Blue: original entries Grey: inserted zeros White border: padding Orange: kernel